Support du cours

M102 : Les bases d'algorithmiques

OUGHZAL OMAR



Introduction à l'algorithmiques

Les bases d'algorithmiques

Notion d'algorithme

Définition

Un algorithme est une suite finie et ordonnée d'instructions permettant de résoudre un problème donné.
Ces instructions doivent être précises, non ambiguës et exécutables.

Un algorithme :

- Reçoit des données d'entrée,
- Effectue un traitement,
- Produit des résultats de sortie.

Rôle de l'algorithmique en programmation

L'algorithmique permet de :

- Analyser un problème avant de le coder,
- Décomposer un problème complexe en étapes simples,
- Choisir les structures de contrôle adaptées,
- Écrire des programmes corrects et compréhensibles.

👉 L'algorithmique est indépendante du langage de programmation.
Un même algorithme peut être implémenté en Python, Java, Kotlin ou tout autre langage.

Méthodologie de résolution de problèmes

Pour résoudre un problème informatique, il est recommandé de suivre les étapes suivantes :

1. Analyser le problème : comprendre l'objectif à atteindre, identifier les contraintes.
2. Identifier les entrées : données fournies par l'utilisateur ou le système.
3. Identifier les sorties : résultats attendus.
4. Décomposer le traitement : séparer le problème en étapes simples, ordonner les actions.
5. Écrire l'algorithme : utiliser une syntaxe claire et normalisée.
6. Tester l'algorithme : vérifier les résultats pour plusieurs cas.
7. Implémenter en Python : traduire l'algorithme en code exécutable.

Environnement de travail

Visual Studio Code (VS Code)

Visual Studio Code (VS Code) est un éditeur de code moderne, gratuit et multiplateforme, largement utilisé dans le domaine du développement et de l'administration système.

VS Code est utilisé comme environnement principal pour :

- Écrire des algorithmes en pseudo-code,
- Écrire et exécuter des scripts Python,

- Écrire des scripts PowerShell et Bash,
- Organiser le travail sous forme de fichiers et de dossiers.

Avantages de VS Code

- Interface claire et ergonomique,
- Support de plusieurs langages (Python, PowerShell, Bash),
- Extensible grâce aux extensions,
- Adapté à l'apprentissage et au monde professionnel.

Pseudo-code (algorithmique)

Le pseudo-code est une manière structurée et simplifiée de décrire un algorithme, en utilisant un langage proche du français, indépendant de tout langage de programmation.

Il permet d'exprimer la logique d'un programme sans se soucier de la syntaxe d'un langage spécifique.

Le pseudo-code est utilisé pour :

- Apprendre à raisonner de manière algorithmique,
- Décomposer un problème en étapes logiques,
- Vérifier la logique d'un algorithme avant de le coder,
- Faciliter la traduction vers Python, PowerShell ou Bash.

👉 Aucun script n'est écrit sans un raisonnement algorithmique préalable.

L'extension vs code Algo Compiler

Algo Compiler est une extension de Visual Studio Code permettant :

- D'écrire des algorithmes en pseudo-code,
- D'exécuter les algorithmes,
- De vérifier la logique avant la programmation...

Python

Python est un langage de programmation interprété, simple et lisible.
Dans ce module :

- Python sert à implémenter les algorithmes,
- La logique est toujours définie d'abord en algorithmique.

Structure générale d'un algorithme

Les commentaires en algorithmique

Un commentaire est un texte explicatif inséré dans un algorithme, non interprété lors de l'exécution.
Il sert uniquement à documenter, expliquer et clarifier le raisonnement.

Les commentaires permettent de :

- Expliquer une étape de l'algorithme,
- Décrire l'objectif d'un bloc d'instructions,

- Faciliter la compréhension pour une autre personne,
- Améliorer la maintenance de l'algorithme.

👉 Un bon algorithme doit être compréhensible même sans exécution.

Syntaxe des commentaires (pseudo-code)

Selon la convention utilisée, un commentaire est généralement précédé par :

```
// commentaire sur une seule ligne

/* commentaire
  sur plusieurs
  lignes */

# commentaire sur une seule ligne avec dièse
```

Structure générale d'un algorithme

Un algorithme respecte la structure suivante :

```
algorithme ExempleAlgorithme // entête de l'algorithme
   // partie déclarative :  Déclaration des variables, constantes, fonctions, procédures
début
   // Le corps de l'algorithme : Instructions à exécuter
fin
```

- L'en-tête : sert à donner un nom à l'algorithme.
- La partie déclarative : sert à déclarer les différents objets (constantes, variables).
- Le corps de l'algorithme : contient les instructions de l'algorithme.

Règles d'écriture et de lisibilité

Pour améliorer la compréhension d'un algorithme :

- Utiliser une indentation claire,
- Choisir des noms explicites pour les variables,
- Ajouter des commentaires si nécessaire,
- Respecter la syntaxe standard.

👉 Un algorithme lisible est plus facile à corriger et à traduire en code.

Exemple simple d'algorithme

```
Algorithme Afficher_Message
Début
  ecrire("Bienvenue en algorithmique")
Fin
```

Traduction Python point par point

```
print("Bienvenue en algorithmique")
```

Démarche Algorithmique → Python

Dans ce module, la démarche suivie sera toujours :

1. Écrire l'algorithme en pseudo-code.
2. Vérifier sa logique avec Algo Compiler.
3. Traduire chaque instruction en Python.
4. Tester le programme Python.

👉 Python n'est jamais utilisé sans algorithme préalable.

Exercice 1

Décrire, sous forme algorithmique, les étapes nécessaires pour :

- allumer un ordinateur,
- ouvrir un navigateur,
- afficher un site web.

Exercice 2

Identifier les entrées, les traitements et les sorties du problème suivant :

Calculer la moyenne de trois notes.

Exercice 3

Écrire un algorithme qui affiche un message de votre choix à l'écran.

Variables et constantes

Définition d'une variable

Une variable est une zone mémoire identifiée par un nom, permettant de stocker une valeur susceptible de changer au cours de l'exécution d'un algorithme.

Chaque variable est caractérisée par :

- un nom,
- un type,
- une valeur.

Définition d'une constante

Une constante est une donnée dont la valeur ne change pas durant l'exécution de l'algorithme.

Syntaxe

```
Variables // ou var
  nom_variable : type
```

```
Constantes // ou const
   NOM : type= valeur
```

Exemple

```
Algorithme Exemple_Variables
Var
   a, b : entier
   note : réel
   nom : chaîne
Const
   ANNEE :entier = 2025
Début
  // corps de l'algorithme
Fin
```

Types de données

Le type de donnée définit la nature des valeurs qu'une variable peut contenir ainsi que les opérations possibles sur cette variable.

Type	Description
Entier	Nombres entiers
Réel	Nombres décimaux
Caractère	Un seul caractère
Chaîne	Suite de caractères
Booléen	Vrai ou Faux

Exemple en algorithmique

```
Algorithme Types_Donnees
Var
  a : entier
  x : réel
  c : caractère
  nom : chaîne
  ok : booléen
Début
   // corps de l'algorithme
Fin
```

Traduction Python

```
a = 10        # int
b = 2.5       # float
c = 'A'       # str (1 caractère)
nom = "Ali"   # str
ok = True     # bool
```

Affectation

L'affectation permet d'attribuer une valeur à une variable.

Syntaxe

```
variable = expression
```

Exemple en algorithmique

```
Algorithme Affectation
Variables x, y : entier
Début
  x = 5
  y =x + 3
Fin
```

Traduction Python

```
x = 5
y = x + 3
```

Entrées et sorties

Les entrées permettent de lire des données, tandis que les sorties permettent d'afficher des résultats.

Syntaxe

```
  Lire(variable)
  Ecrire(expression1, expression2, ...)
```

Exemple en algorithmique

```
Algorithme Lecture_Ecriture
Variables note : réel
Début
  Lire(note)
  Ecrire("Note =", note)
Fin
```

Traduction Python

```
note = float(input())
print("Note =", note)
```

Structure séquentielle

Définition

La structure séquentielle correspond à l'exécution des instructions dans l'ordre où elles sont écrites, sans condition ni répétition.

Exemple en algorithmique

```
Algorithme Structure_Sequentielle
Variables a, b, s : entier
Début
  Lire(a)
  Lire(b)
  s = a + b
  Ecrire("Somme =", s)
Fin
```

Traduction Python

```
a = int(input())
b = int(input())
s = a + b
print("Somme =", s)
```

Exercices

Exercice 1

Écrire un programme qui demande à l'utilisateur deux entiers A et B, qui échange le contenu des variables A et B, puis affiche A et B

Exercice 2

Écrire un algorithme qui lit le prix unitaire PU d'un article, la quantité d'articles Qt et le taux de TVA=20%, et qui fournit le prix total TTC correspondant : $TTC = PU * Qt * (1 + TVA)$

Exercice 3

Écrire un programme qui demande à l'utilisateur le rayon R d'un disque, calcule la surface S et le périmètre P et les affiche à l'écran. $S = \pi R^2$ et $P = 2\pi R$

Exercice 4

Écrire un programme qui demande à l'utilisateur de taper la largeur et la longueur d'un rectangle, puis calcule et affiche son périmètre et sa surface.

Exercice 5

Écrire un programme qui calcule et affiche le volume d'un cylindre après saisie son rayon R et sa hauteur H.tel que $V = H\pi R^2$

Exercice 6

Écrire un programme d'opérations qui calcule la somme, le produit, la soustraction et la division de deux nombres réels saisies par l'utilisateur.

Exercice 7

Écrire un programme qui demande un temps T en secondes, et qui le convertir en heures, minutes et secondes (exemple : $T = 56263 \rightarrow 15 : 37 : 43$)

Exercice 8

Écrire un algorithme qui l'heure H, les minutes M, les secondes S et une durée D en seconde puis affiche l'heure après cette durée. Exemple Pour $H = 20$; $M = 35$; $S = 16$ et $D = 4509$ Affichera le message "$20 : 35 : 35 + 4509\, sec = 21 : 50 : 24$"

Exercice 9

Écrire un programme qui calcule et affiche la distance entre deux points A et b du plan dont les coordonnées (X_a, Y_a) et (X_b, Y_b) sont entrées au clavier. $D = \sqrt{(X_a - X_b)^2 + (Y_a - Y_b)^2}$

Exercice 10

Écrire un programme qui calcule et affiche la somme de quatre entiers entrés au clavier, en utilisant seulement 2 variables.



Structures de contrôle

Les bases d'algorithmiques

Introduction

Dans un algorithme simple, les instructions sont exécutées dans l'ordre.

Exemple :

```
algorithme somme
variables
   a,b,c : entier
debut
   ecrire("donner a :")
   lire(a)
   ecrire("donner b : ")
   lire(b)
   c = a +b
   ecrireln("la somme est ",c)
fin
```

Ce type d'algorithme suit une structure séquentielle.

Cependant, dans la majorité des programmes, il est nécessaire de :

- Prendre des décisions
- Répéter certaines instructions plusieurs fois

Pour cela, on utilise les structures de contrôle.

La notion de condition

Une condition est une expression logique qui peut avoir deux résultats :

- Vrai
- Faux

Les conditions permettent de prendre des décisions dans un algorithme.

Exemple simple de condition

```
age ≥ 18
```

Cette condition signifie :

- Si l'âge est supérieur ou égal à 18 ⟶ la condition est vraie
- Sinon ⟶ la condition est fausse

Opérateurs de comparaison

Les conditions utilisent des opérateurs de comparaison.

Signification	Algorithme	Python
Égal	=	==
Différent	!=	!=
Supérieur	>	>
Inférieur	<	<
Supérieur ou égal	>=	>=
Inférieur ou égal	<=	<=

Exemple

```
note ≥ 10
```

Conditions composées

Une condition peut combiner plusieurs expressions grâce aux opérateurs logiques.

Signification	Algo Compiler	Python
ET logique	ET	and
OU logique	OU	or
NON logique	NON	not

Exemple

```
age ≥ 18 ET age ≤ 60
```

Cette condition signifie que l'âge doit être entre 18 et 60 ans.

Exemple Python

```
age >= 18 and age <= 60
```

Les structures de contrôle

Les structures de contrôle : contrôlent l'ordre d'exécution des instructions dans un programme, permettant de prendre des décisions et de répéter des actions.

Elles permettent :

- D'exécuter une instruction selon une condition
- De répéter une série d'instructions plusieurs fois

Il existe deux grandes catégories :

Type	Rôle
Structure conditionnelle	Prendre une décision
Structure répétitive	Répéter des instructions

Structure conditionnelle simple : Si

La structure conditionnelle Si permet d'exécuter un bloc d'instructions uniquement si une condition est vraie.

Syntaxe algorithmique

```
Si condition Alors
   instructions
FinSi
```

Syntaxe Python

```
if condition:
   instructions
```

Exemple Algorithmique

```
Algorithme TestPositif
Variables
   n : entier
Début
   Lire(n)
   Si n > 0 Alors
      Ecrire "Nombre positif"
   FinSi
Fin
```

Exemple Python

```
n = int(input("Entrer un nombre : "))
if n > 0:
   print("Nombre positif")
```

3. Structure SI / SINON

La structure SI...SINON permet d'exécuter une instruction si la condition est vraie, sinon une autre instruction est exécutée.

Syntaxe algorithmique

```
   Si condition Alors
      instructions
   Sinon
      instructions
   FinSi
```

Syntaxe Python

```
if condition:
   instructions
else:
```

```
   instructions
```

Exemple Algorithmique

```
Algorithme PairImpair
Variables
   n : entier
Début
   Lire(n)
   Si n % 2 = 0 Alors
      Ecrire("Nombre pair")
   Sinon
      Ecrire("Nombre impair")
   FinSi
Fin
```

Exemple Python

```
n = int(input("Entrer un nombre : "))
if n % 2 == 0:
   print("Nombre pair")
else:
   print("Nombre impair")
```

Structure SI / SINON SI

La structure SI...SINON SI permet de tester plusieurs conditions successivement.

Syntaxe algorithmique

```
Si condition1 Alors
   instructions
SinonSi condition2 Alors
   instructions
Sinon
   instructions
FinSi
```

Syntaxe Python

```
if condition1:
   instructions
elif condition2:
   instructions
else:
   instructions
```

Exemple Algorithmique

```
Algorithme Mention
```

```
Variables
   note : entier
Début
   Lire(note)
   Si note >= 16 Alors
      Ecrire("Très bien")
   SinonSi note >= 12 Alors
      Ecrire("Assez bien")
   Sinon
      Ecrire("Passable")
   FinSi
Fin
```

Exemple Python

```
note = int(input("Entrer la note : "))
if note >= 16:
   print("Très bien")
elif note >= 12:
   print("Assez bien")
else:
   print("Passable")
```

Structure SELON (switch)

La structure SELON permet de choisir une action en fonction de la valeur d'une variable.

Syntaxe algorithmique

```
Selon (variable)
   cas valeur1 :
      instructions

   cas valeur2 :
      instructions

   defaut :
      instructions
FinSelon
```

Syntaxe Python

```
match variable:
   case valeur1:
      instructions
   case valeur2:
      instructions
   case _:
```

```
    instructions
```

Exemple Algorithmique

```
Algorithme Menu
Variables
  choix : entier
Début
  Lire choix
  Selon (choix)
    cas 1 :
      Ecrire "Ajouter"
    cas 2 :
      Ecrire "Modifier"
    cas 3 :
      Ecrire "Supprimer"
    defaut :
      Ecrire "Choix invalide"
  FinSelon
Fin
```

Exemple Python

```
choix = int(input("Entrer un choix : "))
match choix:
  case 1:
    print("Ajouter")
  case 2:
    print("Modifier")
  case 3:
    print("Supprimer")
  case _:
    print("Choix invalide")
```

Boucle POUR

La boucle POUR permet de répéter des instructions un nombre déterminé de fois.

Syntaxe algorithmique

```
Pour i de debut à fin Faire
  instructions
FinPour
```

Syntaxe Python

```
for variable in range(debut, fin):
  instructions
```

Exemple Algorithmique

```
Algorithme Compteur
Variables
  i : entier
Début
  Pour i de 1 à 5 Faire
    Ecrire i
  FinPour
Fin
```

Exemple Python

```
for i in range(1, 6):
  print(i)
```

Boucle TANT QUE

La boucle TANT QUE permet de répéter des instructions tant qu'une condition est vraie.

Syntaxe algorithmique

```
TantQue condition Faire
  instructions
FinTantQue
```

Syntaxe Python

```
while condition:
  instructions
```

Exemple Algorithmique

```
Algorithme Compteur
Variables
  i : entier
Début
  i = 1
  TantQue i ≤ 5 Faire
    Ecrire i
    i = i + 1
  FinTantQue
Fin
```

Exemple Python

```
i = 1
while i <= 5:
  print(i)
  i += 1
```

Boucle RÉPÉTER ... JUSQU'À

La boucle Répéter...Jusqu'à exécute les instructions au moins une fois, puis vérifie la condition.

Syntaxe algorithmique

```
repeter
   instructions
jusqua condition
```

Syntaxe Python

Python ne possède pas directement cette structure.

On utilise généralement une boucle while True avec break.

```
while True:
   instructions
   if condition:
      break
```

Exemple Algorithmique

```
Algorithme Saisie
Variables
   n : entier
Début
   Repeter
      Lire(n)
   Jusqua n > 0
Fin
```

Exemple Python

```
while True:
   n = int(input("Entrer un nombre positif : "))
   if n > 0:
      break
```

Instructions de contrôle des boucles en Python

Dans certains cas, il est nécessaire de modifier le comportement d'une boucle pendant son exécution.

Python fournit deux instructions importantes pour cela :

- break
- continue

Ces instructions permettent de contrôler l'exécution des boucles.

Instruction break

L'instruction break permet de quitter immédiatement une boucle, même si la condition de la boucle est toujours vraie.

Elle est souvent utilisée pour :

- Arrêter une boucle lorsqu'une condition est atteinte
- Interrompre une recherche
- Sortir d'une boucle infinie

Syntaxe Python

```
break
```

Exemple

```
for i in range(1, 10):
    if i == 5:
        break
    print(i)
```

Résultat

```
1
2
3
4
```

Lorsque i devient 5, l'instruction break arrête immédiatement la boucle.

Exemple pratique

```
while True:
    mot = input("Entrer le mot de passe : ")

    if mot == "admin":
        print("Accès autorisé")
        break
```

La boucle se répète jusqu'à ce que le mot de passe soit correct.

Instruction continue

L'instruction continue permet de sauter l'itération courante de la boucle et de passer directement à l'itération suivante.

Elle est utilisée pour :

- Ignorer certaines valeurs
- Filtrer des données

Syntaxe Python

```
continue
```

Exemple

```
for i in range(1, 6):
    if i == 3:
        continue
    print(i)
```

Résultat

```
1
2
4
5
```

Lorsque i vaut 3, l'instruction continue ignore cette itération.

Exemple combiné

```
for i in range(1, 10):
    if i == 3:
        continue
    if i == 7:
        break
    print(i)
```

Résultat

```
1
2
4
5
6
```

Bonnes pratiques

Il est recommandé de :

- Utiliser break pour arrêter une recherche
- Utiliser continue pour ignorer certaines valeurs
- Éviter d'utiliser ces instructions de manière excessive afin de garder un code lisible.

Exercices

Exercice 1

Écrire un programme qui demande à l'utilisateur la température T. Puis déterminer l'état de l'eau :

- Solide : si $T \leq 0$
- Liquide : si $0 < T < 100$

- Gaz : si T ≤ 100

Exercice 2

Écrivez un programme qui teste si une note saisie au clavier est comprise entre 0 est 20.

Exercice 3

Écrire un programme qui affiche selon le cas, la nature du caractère (Minuscule, Majuscule, chiffre ou symbole) correspondant à une touche saisie.

Exercice 4

Écrire un algorithme permettant de résoudre une équation du second degré : ax^2 +bx +c = 0

Exercice 5

Écrire un programme calculateur permettant la saisie de deux entiers et une opération $(+, -, /, *)$ et affichant le résultat.

Exercice 6

Écrire un programme qui demande de saisir numéro du mois, puis l'afficher en lettres (1 : janvier, 2 : février, ...)

Exercice 7

Écrire un programme qui détermine si une année saisie au clavier est bissextile. Une année bissextile :

- S'elle est divisible par 400
- Ou S'elle est divisible par 4 et elle n'est pas par 100

Exercice 8

Écrire un programme qui lit le jour J, le mois M et l'année A, puis affiche si la date J/M/A est une date valide

Exercice 9

Écrire un programme qui demande un nombre N, puis calcule et affiche la somme :

$S = 1^2 + 2^2 + 3^2 + \ldots + N^2$

Exercice 10

Écrire un programme qui demande un nombre N, et qui calcule sa factorielle : $N! = 1 \times 2 \times \ldots \times N$

Exercice 11

Écrire un programme qui demande un nombre N puis calcule :

$$S = \begin{cases} 1 + 2 + 3 + \ldots + N, & N \text{ est impair} \\ 2 + 4 + 6 + \ldots + N, & N \text{ est pair} \end{cases}$$

Exercice 12

Écrire un programme qui détermine si un nombre est premier ou non (Un nombre premier n'est divisible que par 1 et par lui-même)

Exercice 13

Écrire un programme qui lit un nombre N, puis affiche si ce nombre est parfait ou non : un nombre est parfait s'il est égal à la somme de ses diviseurs (exemple : $6 = 1 + 2 + 3$ (1,2,3 sont les diviseurs de 6))

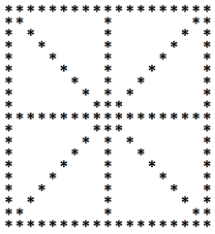
Exercice 14

Ecrire un programme qui demande à l'utilisateur de taper un entier N supérieur à 2, jusqu'à que la réponse convienne, puis qui calcul et affiche tous les termes de la suite de Fibonacci, inférieur ou égaux à N, la suite de Fibonacci est définie comme suite :

$$\begin{cases} U_{n+2} = U_{n+1} + U_n \\ U_1 = 1 \\ U_2 = 1 \end{cases}$$

Exercice 15

Affichez une figure formée d'étoiles de N lignes (N est fourni au clavier) :



Exercice 16

Ecrire un programme qui affiche la table de multiplication pour les nombres 1 à 10

Exercice 17

Ce jeu et très simple, l'ordinateur tire un nombre au hasard entre 1 et 100 et vous avez cinq essais pour le trouver. Après chaque tentative, l'ordinateur vous dira si le nombre entré est trop grand, trop petit, ou si vous avez trouvé le bon nombre

Exercice 18

Écrire un programme qui permet de déterminer la somme des chiffres d'un nombre entier donné. Exemple : pour N= 25418, on aura 2+5+4+1+8 = 20

Exercice 19

Écrire un programme qui demande à l'utilisateur d'entrer un entier, puis le programme trouve et affiche l'image miroir de ce nombre (1234 donne 4321)

Exercice 20

Écrire un programme qui lit le numérateur et dénominateur d'une fraction. Puis la simplifier. Exemple : $\frac{15}{45} = \frac{1}{3}$

Les Fonctions et les procédures

Les bases d'algorithmiques

1. Introduction

Dans les chapitres précédents, les algorithmes sont écrits sous forme de blocs uniques.
Cependant, lorsque les programmes deviennent plus complexes, il est nécessaire de :

- Organiser le code,
- Éviter la répétition,

- Améliorer la lisibilité.

Pour cela, on utilise les fonctions et les procédures.

Fonctions standard (prédéfinies)

Les fonctions standard sont des fonctions déjà disponibles dans le langage.

Elles permettent d'effectuer des opérations courantes sans avoir à les programmer.

Exemples en algorithmique

```
algorithme fonctions
debut
  Ecrireln(longueur("Bonjour"))
  ecrireln(racine(16))
  ecrireln(max(5, 10))
  ecrireln(estChiffre("A"))
  ecrireln(aleatoire(1, 100))
fin
```

Exemples en Python

```
from math import sqrt
from random import randint
print(len("Bonjour"))
print(sqrt(16))
print(max(5, 10))
print("A".isdigit())
print(randint(1, 100)
```

Rôle pédagogique

Les fonctions standard permettent :

- De simplifier les algorithmes,
- De gagner du temps,
- De se concentrer sur la logique.

Notion de fonction

Une fonction est un sous-programme qui permet d'effectuer un traitement spécifique et de retourner une valeur.

Rôle des fonctions

Les fonctions permettent de :

- Structurer un programme,
- Réutiliser du code,
- Améliorer la lisibilité,
- Réduire les erreurs.

4. Syntaxe d'une fonction

Syntaxe algorithmique

```
algorithme NomAlgo
Fonction NomFonction(p1: type, p2: type,….) : type
Début
   instructions
   Retourner valeur
Fin

debut
  // programme principal
fin
```

Syntaxe Python

```
def nom_fonction(parametres):
   instructions
   return valeur
```

Exemple

Algorithmique

```
Fonction somme(a : entier, b : entier) : entier
Début
   Retourner a + b
Fin
```

Python

```
def somme(a, b):
   return a + b
```

Appel (invocation) d'une fonction

L'appel d'une fonction consiste à exécuter cette fonction en lui passant éventuellement des valeurs.

Exemple

Algorithmique

```
ecrire(somme(5,7))
```

Python

```
print(somme(5, 7))
```

☞ La fonction est exécutée au moment de l'appel.

Passage des paramètres

Le passage des paramètres consiste à fournir des valeurs à une fonction lors de son appel.

Types de paramètres

- Paramètres formels : définis dans la fonction
- Paramètres effectifs : utilisés lors de l'appel

Exemple

Algorithmique

```
algorithme fonctions
Fonction somme(a : entier, b : entier) : entier
Début
   Retourner a + b
Fin

debut
  ecrire(somme(5,7))
fin
```

👉 a et b ⟶ paramètres formels

👉 3 et 5 ⟶ paramètres effectifs

Python

```
def somme(a, b):
   return a + b

print(somme(5, 7))
```

Exemple complet de fonction

Algorithmique

```
Algorithme Exemple
Fonction Carre(x : entier) : entier
Début
   Retourner x * x
Fin

Début
   Ecrire(Carre(4))
Fin
```

Python

```
def carre(x):
    return x * x


print(carre(4))
```

Notion de procédure

Une procédure est un sous-programme qui réalise un traitement sans retourner de valeur.

Syntaxe algorithmique

```
Procédure Nom_Procedure(paramètres)
Début
    instructions
FinProcédure
```

Syntaxe Python

```
def afficher():
    print("Bonjour")
```

Organisation d'un programme avec des fonctions

Un programme peut être structuré en plusieurs fonctions :

- Fonction principale
- Fonctions secondaires

Notion de récursivité

Une fonction est dite récursive lorsqu'elle s'appelle elle-même.

Exemple algorithmique

```
Algorithme ExempleRecursif
Fonction Fact(n : entier) : entier
Début
    Si n == 0 Alors
        Retourner 1
    Sinon
        Retourner n * Fact(n - 1)
    FinSi
Fin

Début
    Ecrire(Fact(4))
Fin
```

Exemple python

```
def fact(n):
    if n == 0:
```

```
    return 1
  else:
    return n * fact(n - 1)

print(fact(4))
```

11. Comparaison : fonction vs procédure

Élément	Fonction	Procédure
Retourne une valeur	Oui	Non
Utilisation	Calcul	Action

Exercices

Exercice 1

Créer une fonction qui retourne :

- "Adulte" si âge ≥ 18
- "Mineur" sinon

Exercice 2

Ecrire une procédure ValeurPair(M,N) qui prend deux paramètres M et N et d'afficher toutes les valeurs paires entre M et N si M <

Exercice 3

Écrire une fonction qui permet de calculer la valeur absolue d'un nombre donné.

Exercice 4

Écrivez une fonction qui prend la base et la hauteur d'un triangle et retourne sa surface. Notez que la surface d'un triangle est : (base * hauteur) / 2

Exercice 5

Créer une fonction qui retourne le maximum de trois nombres.

Exercice 6

Écrire une procédure TableMultiplication(N) qui affiche le tableau de multiplication d'un entier positif N passé en paramètre

Exercice 7

Écrire une fonction qui permet de calculer la multiplication de deux nombres A et B entiers en utilisant l'addition.

Exercice 8

Écrire une fonction estPair(N) qui vérifie si un nombre N passé en paramètre est pair ou non.

Exercice 9

Écrire une fonction qui détermine si un nombre est premier ou non (un nombre premier n'est divisible que par 1 et par lui-même)

Exercice 10

Écrire une fonction récursive qui calcule la série de Fibonacci d'un nombre passé en paramétrer.

$$\begin{cases} U_{n+2} = U_{n+1} + U_n \\ U_1 = 1 \\ U_2 = 1 \end{cases}$$

Exercice 11

Écrire une fonction partieEntiere(x) qui retourne la partie entière d'un nombre positif

Exercice 12

Écrire une fonction estBissextile(A) qui prend en paramètre une année A. puis vérifie si cette année est bissextile

Exercice 13

Écrire une fonction dateValide(J,M,A) qui prend en paramètres le jour J, le mois M et l'année A. Puis vérifie si la date est valide



Les Tableaux

Les bases d'algorithmiques

Introduction

Imaginons que, dans un programme, nous ayons besoin d'un grand nombre de variables. Il devient alors difficile de donner un nom à chaque variable.

Exemple 1 :

Écrire un algorithme permettant de saisir cinq notes et de les afficher après avoir multiplié toutes les notes par trois.

Solution :

```
Algorithme Note
Variables N1, N2, N3, N4, N5 : réel
Début
   Écrire("Entrez la valeur de la première note : ")
   Lire(N1)

   Écrire("Entrez la valeur de la deuxième note : ")
   Lire(N2)

   Écrire("Entrez la valeur de la troisième note : ")
   Lire(N3)

   Écrire("Entrez la valeur de la quatrième note : ")
   Lire(N4)

   Écrire("Entrez la valeur de la cinquième note : ")
   Lire(N5)

   Écrireln("La note 1 multipliée par 3 est : ", N1 * 3)
   Écrireln("La note 2 multipliée par 3 est : ", N2 * 3)
   Écrireln("La note 3 multipliée par 3 est : ", N3 * 3)
   Écrireln("La note 4 multipliée par 3 est : ", N4 * 3)
   Écrireln("La note 5 multipliée par 3 est : ", N5 * 3)
Fin
```

La même opération se répète cinq fois. Si l'on voulait réaliser cet algorithme avec 100 notes, cela deviendrait fastidieux.

Comme les variables ont des noms différents, on ne peut pas les parcourir directement à l'aide d'un indice dans une boucle. Cela allonge considérablement le code et le rend très répétitif.

Dans un programme réel, il est souvent nécessaire de manipuler :

- Plusieurs valeurs

- Des ensembles de données
- Des informations structurées

Pour cela, on utilise des structures de données.

Tableaux en algorithmique

Un tableau est une structure permettant de stocker plusieurs valeurs de même type.

Syntaxe algorithmique

```
variables
  T : tableau[Taille] de Type
```

Exemple 2

```
Algorithme Exemple
Variables
  T : tableau[5] de entier
```

Les variables indicées

Définition

Une variable indicée est un élément d'un tableau auquel on accède à l'aide d'un indice.

Contrairement à une variable simple, une variable indicée est identifiée par deux éléments :

- Le nom du tableau : identifie l'ensemble des données.
- L'indice : indique la position de l'élément dans le tableau.

Syntaxe :

```
NomTableau[Indice]
```

Exemple :

Soit le tableau T suivant :

10	20	30	40	50
T[0]	T[1]	T[2]	T[3]	T[4]

Dans ce tableau :

- T[0] représente la première variable indicée, contenant 10.
- T[2] représente la troisième variable indicée, contenant 30.
- T[4] représente la dernière variable indicée, contenant 50.

Chaque variable indicée se comporte comme une variable simple : nous pouvons lire sa valeur, la modifier ou l'utiliser dans des calculs.

Opérations sur les variables indicées

Affectation

L'affectation permet d'attribuer une valeur à un élément du tableau.

```
T[0] = 15
T[2] = 25
```

Après ces instructions, le premier élément contient 15 et le troisième contient 25.

Lecture et affichage

Nous pouvons saisir ou afficher la valeur d'une variable indicée.

```
Lire(T[1])
Ecrire(T[3])
```

- Lire(T[1]) : saisit une valeur et la stocke dans la deuxième case.
- Ecrire(T[3]) : affiche la valeur de la quatrième case.

Opérations arithmétiques

Les variables indicées peuvent être utilisées dans des expressions arithmétiques comme les variables simples.

```
S = T[0] + T[1]
P = T[2] * 3
M = (T[0] + T[1]) / 2
```

Nous pouvons également modifier directement la valeur d'un élément :

```
T[0] ← T[0] + 5
T[1] ← T[1] * 2
```

Comparaison

Nous pouvons comparer les éléments d'un tableau entre eux ou avec une valeur donnée.

```
Si T[0] > T[1] Alors
   Ecrire("Le premier élément est supérieur")
FinSi
```

5. Utilisation d'un indice variable

L'indice d'un tableau peut être une constante, une variable entière ou une expression entière valide.

Exemple :

```
i = 2
ecrireln(T[1]) // indice : entier
Ecrireln(T[i]) // indice : variable entier
Ecrireln(T[i + 2]) // indice : expression valide (entre 0 et taille du tabeau -1)
Pour i de 0 à 4 Faire
   Ecrireln(T[i])
```

```
FinPour
```

Exercice 1

Énoncé : Soit un tableau T contenant cinq nombres entiers. Écrire un algorithme permettant de :

1. Initialiser le tableau avec les valeurs 10, 20, 30, 40 et 50.

2. Remplacer le premier élément par 15.

3. Multiplier le troisième élément par 2.

4. Calculer la somme du deuxième et du quatrième élément.

5. Afficher tous les éléments du tableau.

Solution :

```
Algorithme exercice1
Variables
   T : tableau[5] de entier
   S, i : entier
Début
   T = [10, 20, 30, 40, 50]
   T[0] = 15
   T[2] = T[2] * 2
   S = T[1] + T[3]
   Ecrireln("Somme : ", S)
   Pour i de 0 à 4 Faire
      Ecrireln(T[i])
   FinPour
Fin
```

Opération sur Les Tableaux

Remplir un tableau

```
   Pour i de 0 à 4 Faire
      ecrire("donner T[", i, "] : ")
      Lire(T[i])
   FinPour
```

Initialisation

```
T = [1, 2, 3, 4, 5]
```

Parcours

```
   Pour i de 0 à 4 Faire
      Ecrireln(T[i])
```

```
FinPour
```

Correspondance avec Python

👉 En Python, les tableaux sont remplacés par les listes

Initialisation (simulation d'un tableau statique)

```
# T : tableau[10] de entier
T = [0]*10
```

Les matrices (tableaux à deux dimensions)

Une matrice est un tableau à deux dimensions (lignes et colonnes).

👉 Chaque élément est identifié par deux indices.

Déclaration

```
M : tableau[Nombre Lignes][Nombre colonnes] de type
```

Exemple

```
Algorithme Matrice

Variables
  M : tableau[3][3] de entier
  i, j : entier

Début

  Pour i de 0 à 2 Faire
    Pour j de 0 à 2 Faire
      Lire M[i][j]
    FinPour
  FinPour

  Pour i de 0 à 2 Faire
    Pour j de 0 à 2 Faire
      Ecrire M[i][j]
    FinPour
  FinPour

Fin
```

Traduction en python (simulation)

```
# M : tableau[4][5] de entier
M =[[0]*5]*4
print(M)
```

Exercices

Exercice 1

Écrire un programme qui déclare et remplisse un tableau de 10 valeurs numériques en les mettant toutes à 5.

Exercice 2

Écrire un algorithme qui déclare et remplisse un tableau de N valeurs numériques saisies au clavier. Puis afficher le Tableau. (N est un nombre demandé à l'utilisateur qui ne dépasse pas 100)

Exercice 3

Soit T un tableau de N entiers saisis par l'utilisateur. Écrire un programme calculant la somme, le produit et la moyenne des valeurs d'un tableau

Exercice 4

Soit T un tableau contenant N entiers. Écrire un programme qui parcourt un tableau de nombres entiers et affiche la plus grande valeur présente dans ce tableau.

Exercice 5

Soit T un tableau contenant N entiers. Écrire un programme qui permet d'augmenter d'une valeur X donnée par l'utilisateur toutes les valeurs du tableau.

Exercice 6

Soit T un tableau contenant N notes. Afficher les notes qui sont supérieures à la moyenne du tableau.

Exercice 7

Écrire un programme qui permet de calculer et afficher la somme des valeurs positives, et la somme des valeurs négatives

Exercice 8

Soit T un tableau contenant N entiers. Écrire un programme qui demande à l'utilisateur de saisir un nombre et qui compte combien de fois ce nombre apparaît dans un tableau

Exercice 9

Soit T un tableau contenant N entiers. Ecrire un programme qui permet de chercher la position d'un élément V donné par l'utilisateur T (si V n'existe pas affiche -1).

Exercice 10

Soit T un tableau contenant N entiers. Ecrire un programme qui permet de déterminer et afficher les éléments uniques du tableau.

Exercice 11

Soit T1 et T2 deux Tableaux de N et M Entier. Écrire un programme qui fusionne des de en un seul tableau T3 plus grand

Exercice 12

Soit T un tableau contenant N entiers. On propose d'écrire un programme qui permet d'inverser les éléments de T (permuter T [1] et T[n], puis T [2] et T [n-1], ...).

Exercice 13

Ecrivez un algorithme constituant un tableau, à partir de deux tableaux de même longueur préalablement saisis. Le nouveau tableau sera la somme des éléments des deux tableaux de départ.

Exemple :

Tableau 1 : [4, 8, 7, 9, 1, 5, 4, 6]

Tableau 2 : [7, 6, 5, 2, 1, 3, 7, 4]

Tableau à constituer : [11, 14, 12, 11, 2, 8, 11, 10]

Exercice 14

Écrivez un algorithme remplissant un tableau de 6 sur 13, avec des zéros.

Exercice 15

Écrire un algorithme qui permet :

- De saisir les notes d'une classe de 5 étudiants en 4 matières ;
- De calculer et d'afficher la moyenne de chaque étudiant ;
- De calculer et d'afficher la moyenne de la classe dans chaque matière ;
- De calculer et d'afficher la moyenne générale de la classe.